

Healthcare Insurance Fraud Detection System

Project Progress & Implementation Report

Purpose: Consolidated documentation of the project work completed and discussed so far, covering the frontend, database, backend setup, architecture, current status, and next steps.

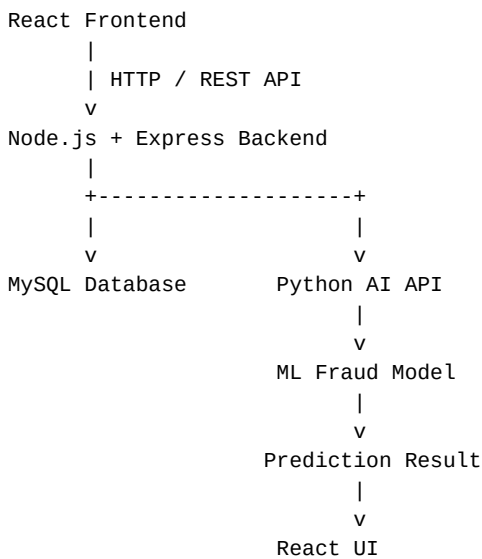
Scope: This report records what has been completed, designed, or planned in the project workflow. Unfinished modules are explicitly marked as pending rather than presented as completed.

1. Project Overview

The Healthcare Insurance Fraud Detection System is being developed as an end-to-end application combining a React frontend, Node.js/Express backend, MySQL database, and a Python-based AI/ML fraud prediction component. The intended system allows users to submit insurance claims, upload supporting documents, analyze claims using an AI model, and view fraud/genuine predictions and risk information.

The project has progressed through frontend UI planning/creation, MySQL installation and database creation, database schema design, and initial Node.js backend setup. The immediate goal is to establish a stable React → Node.js → MySQL flow before completing AI integration and optional blockchain functionality.

2. Overall Architecture



React provides the user interface. Node.js/Express provides REST APIs and application logic. MySQL stores structured data. A Python service is planned to host the machine-learning model and return prediction results to the backend.

3. Frontend Work

A React-based professional healthcare UI was created/planned with separate user and administrator workflows and a dedicated AI fraud-detection area.

Main pages: Login, Register, Forgot Password, Dashboard, Submit Claim, Claim History, Claim Details, Prediction Result, AI Fraud Detection, Profile, Settings, Admin Dashboard, and Not Found/Error page.

Dashboard: Total Claims, Approved Claims, Rejected Claims, Fraud Detected, Pending Claims, Fraud vs Genuine chart, Claims Per Month chart, and Recent Claims table.

Submit Claim: Customer information, policy details, claim amount/type, hospital details, vehicle details where applicable, police report, incident information, description, and uploads for medical bills, FIR, insurance documents, and images.

AI Prediction Result: Fraud/Genuine prediction, confidence/probability, risk level, reasons, recommended action, and probability meter.

Admin Dashboard: Search/filter claims, view users, view predictions, approve/reject claims, delete where appropriate, and display statistics/charts.

Important: the frontend UI is the presentation layer. Any demo/static dashboard numbers should later be replaced by live API responses from the backend and database.

4. Frontend Technology Stack

Technology	Purpose
React 19	Frontend application and components
Vite	Development/build tooling
React Router DOM	Routing and protected routes
Axios	Backend API communication
Bootstrap 5 / Material UI	Responsive UI and styling
React Icons	Icons
Chart.js	Charts and analytics
Context API	Authentication/application state
JWT	Authentication token handling

5. Database Setup

The MySQL environment was installed/configured and the database **insurance_fraud_detection** was successfully created and selected.

```
CREATE DATABASE insurance_fraud_detection;  
USE insurance_fraud_detection;
```

The planned schema contains six core tables: users, policies, claims, documents, ai_predictions, and audit_logs.

5.1 Users Table

Stores user ID, full name, email, phone, password hash, role, creation time, and update time. Roles discussed include user, admin, and inspector.

5.2 Policies Table

Stores policy number, policy type, start/expiry dates, premium, coverage amount, and the relationship to the user who owns the policy.

5.3 Claims Table

The central business table. Stores claim amount, claim type, hospital, vehicle information where applicable, police report status, incident details, description, status, and relationships to user and policy.

5.4 Documents Table

Stores metadata for uploaded medical bills, FIRs, insurance documents, images, and other files. The actual files may be stored locally or in cloud/object storage while MySQL stores metadata and file path information.

5.5 AI Predictions Table

Stores fraud/genuine prediction, fraud probability, genuine probability, risk level, model version, prediction reasons, and prediction timestamp.

5.6 Audit Logs Table

Records important actions such as claim submission, administrative decisions, viewing activity, and AI prediction generation.

6. Backend Setup

A Node.js/Express backend structure was created/planned with separation of configuration, controllers, routes, middleware, models, and services.

```
backend/
|
+-- config/
|   +-- db.js
|
+-- controllers/
|   +-- authController.js
|   +-- claimController.js
|   +-- predictionController.js
|
+-- routes/
|   +-- authRoutes.js
|   +-- claimRoutes.js
|   +-- predictionRoutes.js
|
+-- middleware/
|   +-- authMiddleware.js
|
+-- models/
|
+-- services/
|
+-- .env
+-- server.js
+-- package.json
```

config/db.js: MySQL connection pool. **Controllers:** business logic. **Routes:** API endpoints. **Middleware:** JWT protection. **Models:** database/data-access organization. **Services:** reusable services such as AI communication. **.env:** configuration/secrets. **server.js:** Express entry point.

7. Backend Packages

Package	Purpose
express	Backend REST API framework
mysql2	Node.js to MySQL connection
cors	Cross-origin frontend/backend communication
dotenv	Environment variables
bcrypt	Password hashing
jsonwebtoken	JWT authentication
multer	File uploads
nodemon	Development auto-restart

8. Express Server Testing

A minimal Express server was created with two test endpoints: GET / for a basic response and GET /api/health for a health check.

```
GET /
Healthcare Fraud Detection Backend is Working!

GET /api/health
{"status":"OK"}
```

The backend was initially tested on port 5000. Because a 403 response was observed during browser testing, port 5001 was proposed as an alternative. The exact cause of the 403 issue was not conclusively established in the

discussion, so it remains a troubleshooting item rather than a completed fix.

The server should remain running after node server.js. If the shell prompt immediately returns, the process needs further diagnosis.

9. Environment and MySQL Connection Plan

```
PORT=5001
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=YOUR_MYSQL_PASSWORD
DB_NAME=insurance_fraud_detection
JWT_SECRET=YOUR_SECRET
```

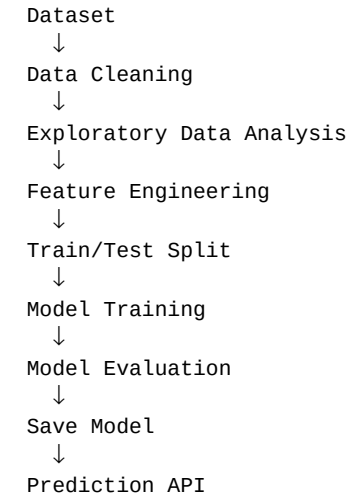
The next database integration step is to implement config/db.js using mysql2 and verify Node.js → MySQL connectivity with a database health-test endpoint.

10. Planned REST API Structure

Area	Endpoint	Purpose
Auth	POST /api/auth/register	Register user
Auth	POST /api/auth/login	Login and return JWT
Auth	GET /api/auth/profile	Get authenticated profile
Claims	POST /api/claims	Create claim
Claims	GET /api/claims	List claims
Claims	GET /api/claims/:id	Get claim details
Claims	PUT /api/claims/:id	Update claim
Claims	DELETE /api/claims/:id	Delete claim
Prediction	POST /api/predictions	Run/store AI prediction
Prediction	GET /api/predictions/:claimId	Get claim prediction
Dashboard	GET /api/dashboard	Return dashboard statistics

11. AI/ML Fraud Detection Plan

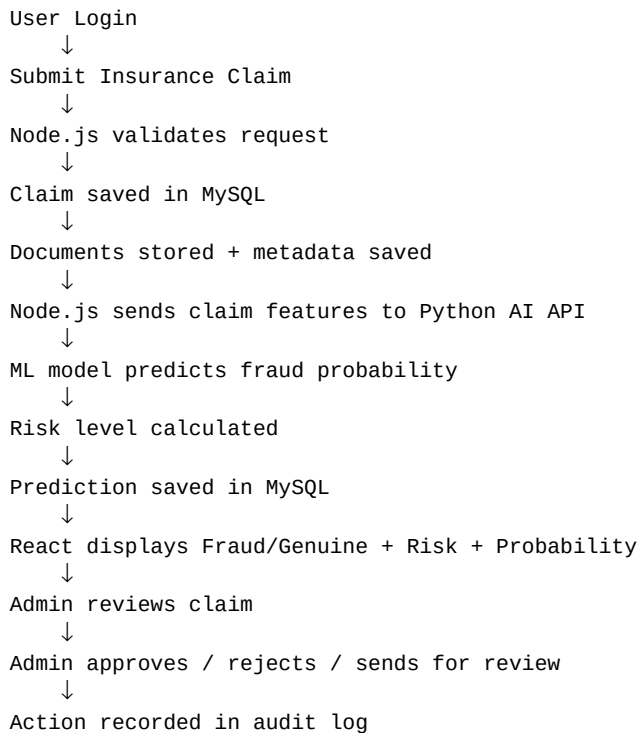
The AI component is intended to analyze claim-related features and predict whether a claim is potentially fraudulent or genuine. The model should provide probabilities and a risk level, with reasons or supporting indicators where possible.



Potential models discussed include Logistic Regression, Decision Tree, Random Forest, and XGBoost. The final model should be selected based on evaluation results rather than assuming one model is best.

A separate Python AI service is planned with training/prediction code, a saved model artifact, requirements.txt, and an API entry point. The Node.js backend will call the Python service, receive the prediction, and store it in ai_predictions.

12. Intended End-to-End Claim Workflow



13. Blockchain Component

The project concept also includes AI + blockchain. Blockchain is planned as a later enhancement, after the core React → Node.js → MySQL → AI workflow is stable. A practical use is recording hashes or verification records for tamper evidence rather than placing sensitive healthcare documents directly on-chain.

Possible flow: claim/prediction data → generate hash → record verification hash/timestamp on blockchain → use the record for integrity verification.

14. Current Project Status

Module	Status
Project concept and architecture	Completed / Defined
React frontend UI	Mostly completed / UI created
AI fraud detection UI	Designed / included
MySQL installation and Workbench	Completed
insurance_fraud_detection database	Created and selected
Database schema	Designed; implementation in progress
Node.js backend structure	Created / defined
Express server	Basic code created; runtime/port issue being tested
MySQL connection from Node.js	Next step

Authentication + JWT	Not yet implemented
Claim APIs	Not yet implemented
File upload	Not yet implemented
AI/ML model	Not yet implemented in current workflow
Python AI API	Not yet implemented
Node → AI integration	Not yet implemented
React → Backend integration	Not yet implemented
Dynamic dashboard	Not yet implemented
Admin backend workflow	Not yet implemented
Blockchain	Planned
Testing	Not yet completed
Deployment	Not yet started

15. Next Steps to Complete the Project

1. Resolve the Express server runtime/port test and confirm the backend remains running.
2. Use port 5001 if port 5000 continues to cause problems.
3. Implement config/db.js and connect Node.js to insurance_fraud_detection.
4. Create and verify database tables in dependency order: users → policies → claims → documents → ai_predictions → audit_logs.
5. Test database connectivity with a dedicated health endpoint.
6. Implement Register API with bcrypt password hashing.
7. Implement Login API and JWT authentication.
8. Protect authenticated routes with authMiddleware.js.
9. Implement claim CRUD APIs.
10. Implement document uploads and document metadata storage.
11. Build and evaluate the fraud detection ML model.
12. Expose the model through a Python API.
13. Connect Node.js to the Python AI service.
14. Connect React to backend APIs using Axios.
15. Replace static frontend values with real database-driven data.
16. Complete Admin workflow and audit logging.
17. Test the full end-to-end claim-to-prediction workflow.
18. Add blockchain integrity functionality only after the core system is stable.
19. Deploy the completed application.

16. Completion Definition

The project will be functionally complete when a user can register and log in, submit an insurance claim with documents, store the claim in MySQL, have the claim analyzed by an ML model, receive a fraud/genuine prediction

with risk information, view the result in React, and allow an authorized administrator to review and update the claim while maintaining an audit trail.

Recommended implementation order: Database → Backend → Authentication → Claims → AI Model → AI API → Integration → Admin → Testing → Optional Blockchain → Deployment.